

Object-Oriented Programming in Python: Classes and Objects

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data in the form of fields (often known as attributes or properties), and code, in the form of procedures (often known as methods). OOP in Python aims to implement real-world entities like inheritance, hiding, polymorphism, etc. in programming. The main focus of OOP is to tie together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Key Concepts of OOP

Classes and Objects:

Class:

A class in Python is like a blueprint for creating objects. A class defines a datatype by combining the fields and methods into a single unit.

Object:

An object is an instance of a class. When class definitions are "instantiated" to create objects, they are equipped with a set of attributes and the ability to perform a set of behaviors via methods.

Attributes and Methods:

Attributes:

Attributes are the variables that belong to a class or object. These variables hold data about our objects.

Methods:

Methods are functions defined inside a class and are used to define the behaviors of an object.

Inheritance:

Inheritance allows new classes to inherit the properties and methods of existing classes. The new class is called a derived (or child) class, and the one from which it inherits is called the base (or parent) class.

Encapsulation:

Encapsulation is the bundling of data with the methods that operate on that data. It restricts direct access to some of an object's components, which can prevent the accidental modification of data.

Polymorphism:

Polymorphism allows the expression of some kind of contract, with potentially many types implementing that contract (whether through class inheritance or not) in different ways, each according to their own purpose.

Defining a Class:

```
In [1]: class Dog:
        # Class Attribute
        species = 'Canis familiaris'

        # Initializer / Instance Attributes
        def __init__(self, name, age):
            self.name = name
            self.age = age

        # instance method
        def description(self):
            return f"{self.name} is {self.age} years old"

        # Another instance method
        def speak(self, sound):
            return f"{self.name} says {sound}"
```

Creating Objects:

Once a class is defined, you can create objects (instances) of that class using the class name followed by parentheses. This process is called instantiation.

```
In [2]: # Instantiate the Dog class
mikey = Dog("Mikey", 6)
```

Accessing Attributes and Calling Methods

Attributes and methods of an object can be accessed using the dot notation (object.attribute or object.method()).

In [3]:

```
# Access the instance attributes
print(f"{mikey.name} is {mikey.age} years old.") # Mikey is 6 years old.

# Is Mikey a mammal?
print(f"Is Mikey a {mikey.species}?") # Is Mikey a Canis familiaris?
```

Mikey is 6 years old.
Is Mikey a Canis familiaris?

Calling Methods:

In [4]:

```
print(mikey.description()) # Mikey is 6 years old
print(mikey.speak("Woof Woof")) # Mikey says Woof Woof
```

Mikey is 6 years old
Mikey says Woof Woof

Class Attributes vs. Instance Attributes

Class Attributes:

Attributes that are shared by all instances of a class. They are defined outside the constructor method.

Instance Attributes:

Attributes that are specific to each instance of a class. They are defined inside the constructor method using self.

In [6]:

```
# Class attribute
print(mikey.species) # Output: Canis familiaris

# Instance attributes
print(mikey.name) # Output: Mikey
print(mikey.age) # Output: 3
```

Canis familiaris
Mikey
6

Encapsulation and Data Hiding

Encapsulation is the bundling of data (attributes) and methods that operate on that data into a single unit (class). In Python, data hiding can be achieved by prefixing attribute names with double underscores (`__`), making them private.

```
In [9]: class Dog:
        def __init__(self, name, age):
            self.__name = name # Private attribute
            self.__age = age   # Private attribute

        def description(self):
            return f"{self.__name} is {self.__age} years old"

mikey = Dog("Mikey", 6)
# print(mikey.__name) # AttributeError: 'Dog' object has no attribute '__n
```

```
-----
-
AttributeError                                Traceback (most recent call las
t)
Cell In[9], line 11
      9 mikey = Dog("Mikey", 6)
     10 # print(mikey.__name) # AttributeError: 'Dog' object has no attri
bute '__name'
----> 11 print(mikey.Dog.__name)

AttributeError: 'Dog' object has no attribute 'Dog'
```

The self Parameter

Purpose of self:

self represents the instance of the class and allows access to the attributes and methods of the class in Python. It binds the attributes with the given arguments. self is not a reserved keyword in Python, it is just a strong convention. However, it is universally followed, and you should do the same to ensure that your code is understandable to other Python programmers.

How self Works:

In Python, the self keyword is used in the method definitions within a class to refer to the instance of the class. When you create a new object from a class, Python automatically passes the object as the first argument to the method being called. This argument is what we refer to as self.

```
In [10]: class Dog:
        def __init__(self, name, age):
            self.name = name
            self.age = age

        def speak(self, sound):
            return f"{self.name} says {sound}"
```

In the Dog class, self.name and self.age are attributes while speak is a method that uses self to access these attributes.

Adding Methods to a Class

Defining Methods:

Methods in a class are functions that define the behaviors of an object. Just like regular functions, methods can take arguments and can return values. Types of Methods:

Instance Methods:

Use `self` to access and modify the state of a specific instance of the class.

Class Methods:

Use `@classmethod` decorator and take `cls` as the first parameter. They can modify the class state that applies across all instances.

Static Methods:

Use `@staticmethod` decorator and do not take `self` or `cls` as the first parameter. They are utility functions that do not modify class or instance state.

```
In [11]: class Dog:
          # Class attribute
          species = 'Canis familiaris'

          def __init__(self, name, age):
              self.name = name
              self.age = age

          def description(self):
              return f"{self.name} is {self.age} years old"

          @classmethod
          def species_info(cls):
              return f"All dogs belong to the species {cls.species}"

          @staticmethod
          def is_dog_friendly():
              return True
```

Displaying Class Attributes and Methods

To access or display class attributes and methods, you can directly use the class name for static and class methods/attributes, or create an instance for instance methods/attributes.

```
In [12]: print(Dog.species_info()) # Displays class method result
print(Dog.is_dog_friendly()) # Displays static method result

mikey = Dog("Mikey", 6)
print(mikey.description()) # Displays instance method result
print(mikey.name) # Accesses instance attribute
```

All dogs belong to the species Canis familiaris
True
Mikey is 6 years old
Mikey

Introduction to the `init()` Method

In Python, the `init()` method is a special method, also known as a constructor or initializer, which is automatically called when a new instance (object) of a class is created. The primary purpose of the `init()` method is to initialize the attributes (variables) of the newly created object with initial values.

Syntax of the `init()` Method

The `init()` method is defined within a class and always has the name `init`. It takes at least one argument, `self`, which refers to the instance being created. Additionally, it can take other arguments required to initialize the object's attributes.

Initializing Instance Attributes

Inside the `init()` method, instance attributes are initialized using the `self` keyword. These attributes can be set to values passed as arguments to the `init()` method or assigned default values within the method.

```
In [13]: class Dog:
def __init__(self, name, age):
    self.name = name # Initialize 'name' attribute
    self.age = age # Initialize 'age' attribute
```

Creating Objects with the `init()` Method

When an object of a class is created, the `init()` method is automatically called, and any arguments passed during object creation are passed to the `init()` method.

```
In [14]: # Creating objects and passing arguments to __init__() method
my_dog = Dog("Buddy", 3)
```

Using self to Access Instance Attributes

Within the `init()` method, and all other instance methods of the class, the `self` keyword is used to access and modify instance attributes.

```
In [15]: class Dog:
          def __init__(self, name, age):
              self.name = name # Initialize 'name' attribute
              self.age = age   # Initialize 'age' attribute

          def description(self):
              return f"{self.name} is {self.age} years old"
```

Benefits of the init() Method

Initialization: Allows for the initialization of instance attributes with initial values. **Instance Specificity:** Ensures that each instance of a class has its own set of attributes with unique values. **Default Values:** Provides default values for attributes, allowing for flexibility when creating objects.

Class Variables

Definition:

Class variables are variables that are shared among all instances of a class. They are defined within a class but outside of any method. **Scope:** Class variables have class-level scope, meaning they are accessible to all instances of the class and the class itself.

Accessing Class Variables:

Class variables are accessed using the class name itself or any instance of the class.

Modifying Class Variables:

Class variables can be modified using the class name. When modified, the change is reflected across all instances of the class.

```
In [16]: class Dog:
          species = 'Canis familiaris' # Class variable

          def __init__(self, name, age):
              self.name = name
              self.age = age
```

Object Variables (Instance Variables)

Definition:

Object variables, also known as instance variables, are variables that are unique to each instance of a class. They are defined within the class's methods and are prefixed with `self`.
Scope: Object variables have instance-level scope, meaning they are accessible only within the instance of the class to which they belong.

Accessing Object Variables:

Object variables are accessed and modified using the `self` keyword within the class's methods

```
In [17]: class Dog:
          def __init__(self, name, age):
              self.name = name # Object variable
              self.age = age   # Object variable
```

Example 2


```
In [18]: class Employee:
# Class variable
raise_amount = 1.05

def __init__(self, name, emp_id, salary):
    self.name = name # Object variable
    self.emp_id = emp_id # Object variable
    self.salary = salary # Object variable

def apply_raise(self):
    # Accessing class variable
    self.salary *= self.raise_amount

# Creating objects
emp1 = Employee("Alice", 1001, 50000)
emp2 = Employee("Bob", 1002, 60000)

# Accessing class variable
print(Employee.raise_amount) # Output: 1.05

# Accessing and modifying object variables
print(emp1.salary) # Output: 50000
emp1.apply_raise()
print(emp1.salary) # Output: 52500 (after applying raise)

# Modifying class variable
Employee.raise_amount = 1.06

# Changes reflected across all instances
print(emp2.salary) # Output: 60000
emp2.apply_raise()
print(emp2.salary) # Output: 63600 (after applying raise with updated clas
```

```
1.05
50000
52500.0
60000
63600.0
```

Introduction to the del() Method

In Python, the **del()** method is a special method, also known as a destructor, which is automatically called when an object is about to be destroyed or deleted. The primary purpose of the **del()** method is to perform any cleanup or resource releasing operations before the object is removed from memory.

Syntax of the del() Method

The **del()** method is defined within a class and always has the name **del**. It does not take any arguments other than **self**, which refers to the instance being destroyed.

```
In [19]: class ClassName:
        def __del__(self):
            # Cleanup or resource releasing operations
            # This method is called when the object is about to be destroyed
```

```
Cell In[19], line 4
      # This method is called when the object is about to be destroyed
```

SyntaxError: incomplete input

When the del() Method is Called

The **del()** method is called when the reference count of an object drops to zero, meaning that there are no more references to the object in the program. This typically occurs when the object is explicitly deleted using the `del` statement, or when the program exits and the object's reference goes out of scope.

Cleanup Operations in the **del()** Method

Resource Release:

The **del()** method is often used to release any resources (such as file handles, network connections, or database connections) that were acquired by the object during its lifetime. This ensures that resources are properly released and prevents resource leaks in the program.

Finalization Tasks:

The **del()** method can also perform any finalization tasks that need to be executed before the object is destroyed. This may include logging, printing messages, or updating state information.

The **del()** method provides a way to perform cleanup and finalization operations before an object is destroyed in Python. By defining this method within a class, you can ensure that any resources acquired by the object are properly released and that any finalization tasks are executed. However, it's important to note that the **del()** method is not always guaranteed to be called, especially in complex or circular reference situations, so it should be used with caution.

Special Functions for Classes in Python

Python provides several special methods, also known as "magic" methods or "dunder" methods (short for "double underscore"), that allow classes to emulate built-in Python behavior and enable custom behavior for operators and functions. Understanding these special methods is crucial for effective object-oriented programming in Python.

1. **str(): String Representation**

The **str()** method returns a string representation of an object when the **str()** function or the **print()** function is called on the object. This method is commonly used to provide a human-readable representation of an object's state.

2. **repr(): Object Representation**

The **repr()** method returns an unambiguous string representation of an object, which should ideally be a valid Python expression that can be used to recreate the object. It is typically used for debugging and logging purposes.

3. **len(): Length of Object**

The **len()** method returns the length of an object when the **len()** function is called on the object. This method is commonly implemented for sequences and collections like lists, tuples, and dictionaries.

4. **getitem(): Indexing and Slicing**

The **getitem()** method allows objects to support indexing and slicing operations, similar to built-in sequences like lists and tuples. It is called when an element is accessed using square brackets (**[]**).

5. **setitem(): Setting Values by Index**

The **setitem()** method allows objects to support assignment of values to elements using square brackets (**[]**). It is called when an element is assigned a value using square brackets (**[]**).

6. **delitem(): Deleting Values by Index**

The **delitem()** method allows objects to support deletion of elements by index using the **del** statement. It is called when an element is deleted using the **del** statement.

7. **iter(): Iterator Protocol**

The **iter()** method returns an iterator object for the class, allowing objects to support iteration. It is commonly used to make custom objects iterable in for loops.

8. **next(): Next Item in Iteration**

The **next()** method is used in conjunction with **iter()** to define custom iteration behavior. It is called to retrieve the next item in the iteration sequence.

9. **contains(): Membership Test**

The **contains()** method allows objects to support membership testing using the **in** and **not in** operators. It is called when the **in** or **not in** operator is used to test for membership.

10. call(): Callable Objects

```
In [ ]: class MyList:
        def __init__(self, data):
            self.data = data

        def __str__(self):
            return str(self.data)

        def __getitem__(self, index):
            return self.data[index]

        def __len__(self):
            return len(self.data)

        def __contains__(self, item):
            return item in self.data

# Creating an object
my_list = MyList([1, 2, 3, 4, 5])

# Using special methods
print(my_list)           # Output: [1, 2, 3, 4, 5]
print(len(my_list))      # Output: 5
print(my_list[2])        # Output: 3
print(3 in my_list)      # Output: True
```

Public and Private Members

In Python, public and private members are used to control access to class attributes and methods. Public members are accessible from outside the class, while private members are accessible only within the class itself. This encapsulation mechanism helps maintain data integrity and prevents accidental modification of sensitive data.

Public Data Members and Methods

Public data members and methods are accessible from outside the class. They are typically used to define the interface of the class and provide access to the class's functionality.

Accessing Public Members:

Public members can be accessed using dot notation (object.member or object.method()).

```
In [ ]: class MyClass:
        def __init__(self):
            self.public_attribute = "Public"  # Public data member

        def public_method(self):
            return "This is a public method"  # Public method
```

Private Data Members and Methods

Definition:

Private data members and methods are accessible only within the class itself. They are typically used to hide implementation details and prevent direct access from outside the class.

Naming Convention:

By convention, private members and methods are prefixed with double underscores (__).

```
In [ ]: class MyClass:
        def __init__(self):
            self.__private_attribute = "Private"  # Private data member

        def __private_method(self):
            return "This is a private method"  # Private method
```

Name Mangling:

Python performs name mangling on private members, prefixing the member name with `__classname` to make it less accessible from outside the class.

```
In [ ]: class MyClass:
        def __init__(self):
            self.__private_attribute = "Private"  # Name mangling: _MyClass__pr
```

Accessing Private Members and Methods

Accessing Within the Class:

Private members and methods can be accessed from within the class itself.

```
In [ ]: class MyClass:
        def __init__(self):
            self.__private_attribute = "Private"

        def access_private(self):
            return self.__private_attribute
```

Name Mangling:

Private members and methods can be accessed from outside the class using name mangling, although it is not recommended.

```
In [ ]: obj = MyClass()
print(obj._MyClass__private_attribute)  # Accessing private member using na
```

Benefits of Encapsulation

Data Hiding:

Encapsulation hides the internal state of objects, preventing direct access and modification.

Modularity:

Encapsulation promotes modularity by separating implementation details from the interface of a class.

Security:

Encapsulation improves security by controlling access to sensitive data and operations.

Example

```
In [ ]: class BankAccount:
    def __init__(self, account_number, balance):
        self.__account_number = account_number
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient funds")

    def get_balance(self):
        return self.__balance

# Creating an object
account = BankAccount("1234567890", 1000)

# Accessing public methods
account.deposit(500)
account.withdraw(200)

# Accessing private data member (not recommended)
print(account._BankAccount__balance)  # Output: 1300
```

Calling a Class Method from Another Class Method

Syntax:

To call a class method from another class method, use the cls parameter to refer to the class itself and invoke the method using the class name.

```
In [ ]: class MyClass:
        @classmethod
        def class_method1(cls):
            print("Class method 1")

        @classmethod
        def class_method2(cls):
            cls.class_method1() # Calling class method 1 from class method 2
            print("Class method 2")
```

Using cls Parameter:

The cls parameter is a convention used to refer to the class itself within class methods. It behaves similarly to the self parameter used in instance methods but refers to the class instead of an instance.

Accessing Class Methods:

Class methods can be accessed using dot notation (cls.method()), where cls is a reference to the class itself.

```
In [ ]: class MathOperations:
        @classmethod
        def add(cls, a, b):
            return a + b

        @classmethod
        def multiply(cls, a, b):
            return a * b

        @classmethod
        def add_and_multiply(cls, a, b):
            sum_result = cls.add(a, b) # Calling class method 'add'
            product_result = cls.multiply(a, b) # Calling class method 'multip
            return sum_result, product_result

# Calling class method 'add_and_multiply'
result = MathOperations.add_and_multiply(3, 4)
print("Sum:", result[0]) # Output: Sum: 7
print("Product:", result[1]) # Output: Product: 12
```

Benefits of Calling Class Methods from Other Class Methods

Code Reusability:

Calling class methods from other class methods promotes code reuse, allowing common functionality to be shared across different parts of the class.

Modular Design:

By organizing functionality into class methods, classes become more modular and easier to understand, maintain, and extend.

Encapsulation:

Class methods encapsulate related functionality within the class, improving encapsulation and separation of concerns.

Inheritance

Inheritance is a fundamental concept in object-oriented programming that allows classes to inherit attributes and methods from other classes. It enables code reuse and promotes modular design by establishing a hierarchical relationship between classes. In Python, inheritance is implemented using the syntax `class DerivedClassName(BaseClassName):`

Types of Inheritance Single Inheritance: A derived class inherits from only one base class.

Multiple Inheritance: A derived class inherits from multiple base classes. Multilevel

Inheritance: A derived class inherits from a base class, which itself is derived from another base class. Hierarchical Inheritance: Multiple derived classes inherit from a single base class.

Vehicles

Consider a scenario where you want to model different types of vehicles in Python. You can create a base class `Vehicle` with common attributes and methods shared by all vehicles, such as `make`, `model`, and `fuel_type`. Then, you can create derived classes for specific types of vehicles, such as `Car`, `Truck`, and `Motorcycle`, which inherit from the `Vehicle` class and may have additional attributes or methods specific to each type.


```
In [ ]: class Vehicle:
    def __init__(self, make, model, fuel_type):
        self.make = make
        self.model = model
        self.fuel_type = fuel_type

    def drive(self):
        print("Vehicle is driving...")

class Car(Vehicle):
    def __init__(self, make, model, fuel_type, num_doors):
        super().__init__(make, model, fuel_type)
        self.num_doors = num_doors

    def honk(self):
        print("Car is honking...")

class Truck(Vehicle):
    def __init__(self, make, model, fuel_type, payload_capacity):
        super().__init__(make, model, fuel_type)
        self.payload_capacity = payload_capacity

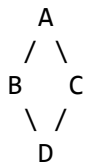
    def load_cargo(self):
        print("Truck is loading cargo...")

class Motorcycle(Vehicle):
    def __init__(self, make, model, fuel_type, has_sidecar):
        super().__init__(make, model, fuel_type)
        self.has_sidecar = has_sidecar

    def wheelie(self):
        print("Motorcycle is performing a wheelie...")
```

The Diamond Problem

The diamond problem is a common issue in languages that support multiple inheritance, such as Python. It occurs when a class inherits from two or more classes that have a common ancestor. If the common ancestor's methods or attributes are overridden in the derived classes, it may lead to ambiguity and conflicts in method resolution.



In the above diagram, classes B and C inherit from class A, and class D inherits from both classes B and C. If class B and class C both override a method from class A, it becomes ambiguous which version of the method class D should inherit. This ambiguity can lead to unexpected behavior and is known as the diamond problem.

Resolving the Diamond Problem in Python

Python uses a method resolution order (MRO) algorithm called C3 linearization to resolve the diamond problem. It follows a specific order to determine the inheritance hierarchy and